

1. Use Bounded Type Parameters with extends (Beginner Level)

What are Bounded Type Parameters?

A **bounded type parameter** restricts the types that can be used with a generic class or method.

It uses the **extends** keyword.

```
class Box {  
    T value;  
}
```

Here, T can only be:

- Integer
 - Double
 - Float
 - Long
 - Any class that extends Number
-

Example

```
class Calculator {  
  
    public double square(T num) {  
        return num.doubleValue() * num.doubleValue();  
    }  
}
```

Using it:

```
Calculator c1 = new Calculator<>();  
Calculator c2 = new Calculator<>();
```

✗ Invalid

```
Calculator c3 = new Calculator<>();
```

Compile-time error because String is **not** a subclass of Number.

Why Use Bounded Generics?

They ensure that only appropriate types are accepted.

Without bounds:

```
class Box {  
}
```

Anything is allowed.

With bounds:

```
class Box {  
}
```

Only numeric types are allowed.

Important Interview Points ★

- extends is used to **restrict generic types**.
- A bounded type parameter accepts only the specified class or its subclasses.
- Bounds provide **compile-time type safety**.
- They also allow access to methods of the upper bound.

Example:

```
T value;  
  
value.doubleValue(); // Allowed because T extends Number
```

- A generic type can have **multiple bounds**.

Example:

>

(If multiple bounds are used, the class must come first, followed by interfaces.)

Tricky Interview Questions

Q1. Why use extends in generics?

Answer:

To restrict the allowed types and improve type safety.

Q2. Can String be used with T extends Number?

Answer: No.

String does not extend Number.

Q3. Why is the keyword extends used even for interfaces?

Answer:

In generics, extends means "**is a subtype of**", so it works for both classes and interfaces.

Q4. Can we specify multiple bounds?

Answer: Yes.

>

Q5. What advantage do bounded generics provide besides restriction?

Answer:

They allow calling methods defined in the upper bound.

Interview Summary

- **Bounded type parameters** restrict which types can be used with generics.
- They use the **extends** keyword.
- Bounds improve **type safety** and allow access to methods of the upper bound.
- Multiple bounds are supported using &.
- They are commonly used in generic libraries and collection utilities.

Interview One-Liner:

"Bounded type parameters use extends to restrict generic types, providing compile-time safety and access to methods of the specified upper bound."

2. Understand Wildcard ? Concept for Flexible APIs (Interview Notes)

What is a Wildcard (?)?

A **wildcard (?)** represents an **unknown type** in generics.

It allows methods to work with **different generic types** without knowing the exact type.

```
List list;
```

Here, list can refer to:

- List
- List
- List

and many others.

Example

```
public static void print(List list) {  
    for (Object obj : list) {  
        System.out.println(obj);  
    }  
}
```

Using it:

```
print(List.of("Java", "Spring"));  
  
print(List.of(10, 20, 30));
```

The same method works for different list types.

Why Use Wildcards?

Without wildcards:

```
void print(List list)
```

Only List is accepted.

With wildcards:

```
void print(List list)
```

Any type of list is accepted.

Types of Wildcards (Beginner Level)

1. Unbounded Wildcard

```
List
```

Accepts any generic type.

2. Upper Bounded Wildcard

List

Accepts:

- List
- List
- List

3. Lower Bounded Wildcard

List

Accepts:

- List
 - List
 - List
- (Beginner interviews usually focus mainly on ? and ? extends.)

Important Interview Points

- ? represents an **unknown type**.
- Wildcards make APIs **more flexible and reusable**.
- List can reference a list of **any type**.
- With List, you can safely **read** elements as Object.
- You generally **cannot add** elements (except null) because the actual type is unknown.
- ? extends T is mainly used for **reading** data.
- ? super T is mainly used for **writing** (adding) data.

Tricky Interview Questions

Q1. What does ? represent?

Answer:

An unknown generic type.

Q2. Can we add elements to List?

Answer: No.

Except for null, because the compiler doesn't know the actual element type.

Q3. Can we read elements from List?

Answer: Yes.

They are returned as Object.

Q4. Difference between and ?

Answer:

- declares a **new type parameter**.
 - represents an **existing unknown type**.
-

Q5. Why are wildcards useful?

Answer:

They make methods work with multiple generic types without sacrificing type safety.

Q6. What is the PECS rule?

Answer:

A common interview rule:

- **Producer → extends**
- **Consumer → super**

If a collection **produces** data, use extends.

If it **consumes** (accepts) data, use super.

Interview Summary

- **Wildcards (?) make generic methods more flexible by accepting different generic types.**
- List works with lists of any type.
- ? extends T is used when reading from a collection.
- ? super T is used when writing to a collection.

- Wildcards are widely used in the Java Collections Framework to create reusable APIs.

Interview One-Liner:

"Wildcards (?) allow generic APIs to accept multiple parameterized types safely, making code more flexible and reusable while preserving type safety."